

Experiment No. 3

Title: To practice Pair Programming through writing simple code. **Problem Description:** Students are expected to for pair in which one writes the code and other improve the quality. Next hour students are expected to exchange the role. The following programs can be used to practice Pair Programming (PP).

- ❖ Menu driven program to implement stack operation or your mini project.
- ❖ To find GCD of the two numbers or any one feature related to your mini project.
- ❖ To check whether a given string is palindrome or any second feature related to your mini project.
- ❖ Pattern matching program or any third feature related to your mini project.
- ❖ Find a pair of strings of numbers that do not exceed a given limit or any other feature related to your mini project.

Aim:

To understand and practice Pair Programming methodology by alternating roles of Driver and Navigator while developing simple programs.

Step 1: Forming the Pair

1. Students form a team of two members.
2. Decide initial roles:
 - ❖ **Driver** – Writes the code.
 - ❖ **Navigator** – Reviews, suggests improvements, checks logic and syntax.

Step 2: Understanding the Problem

3. Read and understand the given problem clearly.
4. Discuss the logic, algorithm, and possible edge cases before coding.
5. Write the algorithm or flowchart in the observation notebook.

Step 3: First Coding Session

6. The Driver starts writing the code.
7. The Navigator:
 - ❖ Reviews each line of code.

- ❖ Suggests improvements.
 - ❖ Identifies errors.
 - ❖ Ensures proper indentation and meaningful variable names.
8. Test the program with sample inputs.
 9. Debug and correct errors collaboratively.

Step 4: Role Exchange

10. After some times, students exchange roles.
 - ❖ Previous Navigator becomes Driver.
 - ❖ Previous Driver becomes Navigator.
11. Continue coding another program or improve the existing program.
12. Add enhancements such as:
 - ❖ Input validation
 - ❖ Error handling
 - ❖ Better user interaction

Step 5: Testing and Validation

13. Execute the program with different test cases.
14. Verify correctness of output.
15. Take the screen shots from Driver window and Navigator window.
16. Record sample input and output in the notebook.

Step 6: Code Quality Review

16. Check:
 - ❖ Readability of code
 - ❖ Proper comments
 - ❖ Modularity (use of functions)
 - ❖ Efficiency
17. Make necessary refinements.

Step 7: Conclusion

18. Write observations about:
 - ❖ Benefits of pair programming
 - ❖ Challenges faced

❖ Learning outcomes.

Result:

The programs were implemented successfully using Pair Programming, resulting in improved code quality, better collaboration, and effective error detection.